Optimization of Premium Investment Portfolios

Capgemini – The Hartford Challenge

Luke Sebold · Binayek Tiwari · Iraj Nelson

YQuantum Hackathon 2026 | Supported by QuEra, Capgemini Quantum Lab, The Hartford

Introduction

Context: *Insurance companies like* ***The Hartford*** *manage massive premium portfolios where risk mitigation is as critical as ROI.*

The Constraint: *Real-world portfolios often require binary decision-making (fully in or fully out), creating a combinatorial* ***"hard" problem*** *for* ***classical solvers****.*

Our Approach: *We used an* ***innovative approach*** *to solve the quantum problem and added an* ***additional layer*** *to make our solution* ***real***

Mathematical Formulation

From classical portfolio theory to quantum hardware

$$\min_{w} \quad q \sum_{i} \sum_{j} w_i w_j \sigma_{i,j} - \sum_{i} w_i \mu_i$$

1 Markowitz Objective → **2** QUBO Formulation → **3** Ising Hamiltonian → **4** Unitary Operator

$$\min_{w} \quad q \sum_{i,j} w_i \sigma_{ij} w_j - \sum_{i} \mu_i w_i + \lambda \left(\sum_{i} w_i - B \right)^2$$

$$H(\sigma) = \sum_{i<j} J_{ij}\, \sigma_i \sigma_j + \sum_{j} h_j\, \sigma_j$$

$$H(w) = \sum_{i<j} J_{ij} w_i w_j + \sum_{i} h_i w_i + \text{constant},$$

$$J_{ij} = 2\left(q\sigma_{ij} + \lambda\right), \qquad \forall i < j, \qquad h_i = q\sigma_{ii} + \lambda - \mu_i - 2\lambda B.$$

We transform the classical Markowitz portfolio optimization into a QUBO problem, map it to an Ising Hamiltonian, and derive the unitary operator for quantum execution via QAOA.

From Hamiltonian to Unitary Operator

$$H = \sum_{i<j} J_{ij} Z_i Z_j + \sum_i h_i Z_i$$

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

$$Z_i = I^{\otimes(i-1)} \otimes Z \otimes I^{\otimes(n-i)}.$$

$$Z_i Z_j = I^{\otimes(i-1)} \otimes Z \otimes I^{\otimes(j-i-1)} \otimes Z \otimes I^{\otimes(n-j)}.$$

$$Z|0\rangle = |0\rangle,$$
$$Z|1\rangle = -|1\rangle,$$

$$E(x_1, \ldots, x_n) = \sum_{i<j} J_{ij}\, s_i s_j + \sum_i h_i\, s_i.$$

$$s_i = 1 - 2x_i,$$

$$U(\gamma) = e^{-i\gamma H}$$

$$U_{kk}(\gamma) = \exp\left(-i\gamma\left(\sum_{i<j} J_{ij}\, s_i^{(k)} s_j^{(k)} + \sum_i h_i\, s_i^{(k)}\right)\right)$$

Computing Energies for n Qubits

For each basis state k (from 0 to $2^n - 1$), write k in binary to get $x_1x_2...x_n$, convert to spins $s_i = 1 - 2x_i$, then compute:

$$E_k = \sum_{i<j} J_{ij} s_i^{(k)} s_j^{(k)} + \sum_i h_i s_i^{(k)}$$

Verification: n = 2 qubits

k	Binary	s_1, s_2	Energy E_k
0	$\|00\rangle$	+1, +1	$J_{12} + h_1 + h_2$
1	$\|01\rangle$	+1, −1	$-J_{12} + h_1 - h_2$
2	$\|10\rangle$	−1, +1	$-J_{12} - h_1 + h_2$
3	$\|11\rangle$	−1, −1	$J_{12} - h_1 - h_2$

Hardware Optimization

Pauli Propagation

For gate matrices with lots of empty terms (or near zero terms) how can we optimize?

- Encode our gates with only the significant terms.
- Set a minimum threshold of significance and a maximum number of allowed terms.

$$\begin{bmatrix} a_{11} & 0 & 0 & \cdots & 0 \\ 0 & a_{22} & 0 & \cdots & 0 \\ 0 & 0 & a_{33} & \cdots & 0 \\ \vdots & \vdots & \vdots & & \vdots \\ 0 & 0 & 0 & \cdots & a_{nn} \end{bmatrix}$$

Results: Classical vs Quantum

Classical Solution

$$\min q \sum_i \sum_j w_i w_j \sigma_{ij} - \sum_i w_i \mu_i$$

$$s.t. \sum_i w_i = 1$$

```
Best binary portfolio (B=4) at each risk aversion level:
--------------------------------------------------------------------------------
q=  0.5: return=8.41%  vol=12.07%  Sharpe=0.697
         Assets: A004, A005, A016, A030
q=  1.0: return=8.21%  vol=10.34%  Sharpe=0.794
         Assets: A001, A004, A016, A006
q=  2.0: return=8.21%  vol=10.34%  Sharpe=0.794
         Assets: A001, A004, A016, A006
q=  5.0: return=8.06%  vol=9.98%  Sharpe=0.807
         Assets: A004, A005, A030, A006
q= 10.0: return=8.06%  vol=9.98%  Sharpe=0.807
         Assets: A004, A005, A030, A006

Max Sharpe binary portfolio (B=4):
  Return=8.06%, Vol=9.98%, Sharpe=0.807
  Selected (4): A004, A005, A030, A006
```

Quantum (QAOA) Solution

$$\min_w \quad q \sum_{i,j} w_i \sigma_{ij} w_j - \sum_i \mu_i w_i + \lambda \left(\sum_i w_i - B \right)^2$$

```
Computing objective values for comparison...
Average objective over all 256 bitstrings: 0.005244
Minimum objective over all bitstrings: -0.302817 (bitstring |10001101|)

Top 10 bitstrings: objective vs average
------------------------------------------------------------
 1. |01010110| -> obj=-0.194892, diff_vs_avg=-0.200136, p=0.0160
 2. |01100011| -> obj=-0.127140, diff_vs_avg=-0.132384, p=0.0140
 3. |10101001| -> obj=-0.269344, diff_vs_avg=-0.274587, p=0.0130
 4. |00100111| -> obj=-0.164343, diff_vs_avg=-0.169587, p=0.0120
 5. |00110110| -> obj=-0.172356, diff_vs_avg=-0.177600, p=0.0120
 6. |01101010| -> obj=-0.140954, diff_vs_avg=-0.146198, p=0.0120
 7. |01110010| -> obj=-0.131469, diff_vs_avg=-0.136713, p=0.0120
 8. |00101110| -> obj=-0.173251, diff_vs_avg=-0.178494, p=0.0110
 9. |00111010| -> obj=-0.141822, diff_vs_avg=-0.147066, p=0.0110
10. |10010110| -> obj=-0.213579, diff_vs_avg=-0.218822, p=0.0110
```

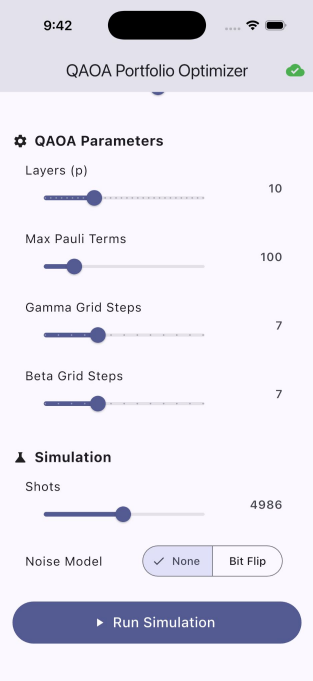
Code to Reality

*We've bridged the gap between our **powerful algorithms** to **actual companies** like The Hartford.*

Mobile App Interface

Python scripts work, but aren't useful for a real company.

- **Cross-Platform App** with Flutter and Dart
  - Complete control over parameters
  - Detailed graphics
- **Python / Flask Backend** for quantum analysis
  - REST API loads data, runs our algorithm
  - Runs behind the scenes
  - Communicates with frontend app
- **Fully installable**
  - **eduroam** is the our only blocker from all running simulations right now



Conclusion & Next Steps

Overall success!

Given more time, we would have liked to focus more on the other information available in the asset data, tune our gamma and beta values for QAOA to more efficiently find our minima, and experiment with different penalty values and layers of operators.

Thank you for your time

Luke Sebold · Binayek Tiwari · Iraj Nelson

YQuantum 2026 | Capgemini – The Hartford Challenge